

面试文档（面向面试·第一人称话术版）

—— 姚凯锐

🎯 **终面模式（Final Mode）预告：**全部话术过关+题库自查全绿后，用户说「开始终面模式」→ 全真模拟一场完整面试（自我介绍→连环深挖→八股→手撕→场景题→HR→反问，全程不给提示），结束后出《评分报告》+薄弱点回炉清单。详见 [面试会话记忆.md](#) 第六节。

⚠️ 本文件使用规范（专门会话必读，最后更新 2026-09-04）

1. **定位：**面向面试的话术文档，场景=把这些内容**讲给面试官听**（第一人称口语化），不是学习笔记
2. **生产方式：**AI 代拟初稿（基于真实源码+面经）→ 用户通读 + 逐节脱稿复述 → AI 口头拷打
3. **顺序：**项目1 ESP32终端 → 项目2 ROS2小车 → 实习 → 项目3 工业互联；**滚动式：一个项目过关再下一个**
4. **题库：**`C:\Users\26476\Desktop\八股文复习\项目面试问题清单.md`（136题v2，带追问链）——考核从这里抽题
5. **事实纪律：**话术技术细节必须和源码对得上（源码：`C:\EspressIf\frameworks\esp-idf-v5.5.5\workspace\` 各章工程 + `workspace\ESP32-S3智能终端开发教程(ESP-IDF).md` 教程原文）；**不确定的宁可写“欠账/下一步计划”，不吹**
6. **考核方式（2026-09-04 改版）：**不再写答题卷。用户通读 2.1~2.6 → 逐节脱稿复述 → 自觉差不多时说「开始考」→ AI 口头抽题+追问链刨根问底 → 卡壳处回炉 → 每次考核在 2.8 记一行
7. **Typora 纪律：**AI 写本文件期间，你开着 Typora 的旧缓冲一旦点保存，会把 AI 写入的内容整个覆盖回去。所以：**AI 说“改完了”你再关标签页重开；改的过程别碰保存**
8. **话术格式约定（怎么读这份文档）：**
 - **正文「照着说」**=口语，直接背；技术名词（FreeRTOS/LVGL/I2S/DMA/NVS/SNTP/cJSON/OPUS/PDM/互斥锁/双核）面试本来就要说，保留
 - **（括号里的函数名/宏名）=证据牌：**说的时候跳过括号；面试官追问“具体怎么实现的、什么接口”时，再把括号里的名字报出来
 - 🛠️ **【源码依据】**=这条答案背后的真实函数/配置/文件，自己核对用，永远不用念出口

📐 每个项目的写作模板（照老师样例）

- x.1 项目介绍（30秒版）—— 一段话：项目是什么+核心功能+技术栈
- x.2 项目介绍（1分钟版）—— 加架构/我做了什么/难点概述
- x.3 架构与分层 —— 硬件/软件分层/数据流向（面试官会让你画或口述）
- x.4 功能列表 —— **bullet + 每条细节展开**（像老师写行车记录仪三模式那样）
- x.5 分工与设计 —— 职责+设计理念（高内聚低耦合等）+ 对外接口/模块清单
- x.6 技术专题深挖 —— 每专题 = 预判问题 + 第一人称回答话术
- x.7 项目成果 —— 量化数据
- x.8 模拟面试记录 —— 滚动追加（日期/被问倒的题/回炉结果）

一、自我介绍

1.1 一分钟版（初稿·待用户复述修改）

面试官您好，我叫姚凯锐，广州南方学院电气工程及其自动化专业。我有两段主要经历：一是在深圳百问网科技做嵌入式软件开发实习，负责串口转WiFi模块的驱动开发，基于STM32F407+ESP8266+FreeRTOS，做了AT指令封装成标准Socket接口、DMA+IDLE方案解决高波特率丢包这些问题。此外我独立完成了三个项目：基于libmodbus和IAP固件升级的工业互联设备管理系统、基于ESP32-S3+LVGL的智能终端、以及树莓派ROS2的SLAM导航小车。我熟悉C语言、FreeRTOS、常用通信协议，求职意向是嵌入式软件开发岗位。

1.2 三分钟版

（待写：一分钟版+项目各展开一段+个人特质收尾）

二、项目1：ESP32-S3 智能终端（第一个做，进行中）

简历项目描述（改稿·按真实工程写准，你改简历直接抄这段）

独立设计并实现基于 ESP32-S3（双核 LX7 @160MHz，16MB Flash + 8MB PSRAM）的智能终端：移植 LVGL 8.3 并通过乐鑫官方适配层完成 ST7789 SPI 屏与 XPT2046 触摸的显示/输入对接，配置 PSRAM 全屏双缓冲，解决多任务下图形库线程安全；基于 FreeRTOS 双核调度，音频混音/解码任务绑核 Core1、应用任务绑核 Core0；实现屏上直连 WiFi 配网（凭据 NVS 持久化、重启免配置）、SNTP 网络对时、天气 API + cJSON 解析渲染；SD 卡（SPI + FATFS 中文长文件名）MP3 播放：libhelix 解码、环形缓冲、混音器、I2S DMA 输出；集成小智语音 AI 对话：PDM 麦克风采集、OPUS 编解码、WebSocket 云端链路，TTS 与音乐经混音器共用喇叭输出；另含 2048/羊了个羊/消消乐/植物大战僵尸四款 LVGL 游戏与串口助手。

2.1 项目介绍（30秒版·初稿）

这个项目是我独立完成的智能终端设备，基于ESP32-S3双核芯片（160MHz），外挂16MB Flash和8MB PSRAM。它实现了一个完整的嵌入式产品形态：LVGL图形界面可以显示天气和时间，WiFi配网后自动同步NTP(网络时间协议)时间、拉取天气API并解析JSON(轻量级文本数据交换格式)渲染到屏幕，本地通过SPI读SD卡里的MP3文件、经I2S输出音频播放，还集成了语音AI对话功能（PDM麦克风采集+OPUS编解码+云端大模型）。整个系统跑FreeRTOS，双核分工，我完成了LVGL到这块SPI屏的完整移植适配——显示缓冲、触摸对接、多任务线程安全都是自己调的，是把我显示驱动、RTOS多任务、网络、音频这一整套技能串起来的项目。

2.2 项目介绍（1分钟版·已核对源码）

（30秒版压缩进开头两句）架构上我分了三层：最底下驱动层——屏、触摸、SD卡、I2S功放、PDM麦克风这些硬件驱动；中间层——LVGL图形库，通过乐鑫官方的适配组件接到驱动上，再加上文件系统、WiFi和网络组件，还有我集成的一个音频混音库；最上面应用层——十个界面加一组后台任务：WiFi、时间天气、音乐控制、语音对话四个应用任务，加LVGL渲染任务和三个音频任务（混音、MP3解码、TTS流）。任务之间用全局状态结构体加标志位通信，谁要碰界面先拿图形库的互斥锁。这是我独立完成的项目：从点亮屏、移植驱动、划分任务，到全部应用逻辑。最难的两块：一是多任务环境下图形库的线程安全；二是音乐和AI语音要共用一个喇叭——我用混音器把两路合成一路，做到放着歌也能随时跟AI说话。做完它，我把显示驱动、RTOS、网络、音频四条技能线真正串成了一个完整产品。

🔗【源码依据】lvgl_set.c（适配层初始化）/ task_set.c（任务编排）/ ui_set（十个界面）

2.3 架构与分层（已核对源码·照着说）

- **硬件（说）**：主控是 ESP32-S3 双核（主频我配的 160MHz）；屏是 ST7789 驱动的 240×320 SPI 彩屏，横屏用、16 位色、SPI 跑 40MHz、背光用 PWM 调亮度；触摸是 XPT2046 电阻屏；SD 卡走 SPI；喇叭接 I2S 数字功放；录音用 PDM 数字麦克风——所以这台设备既能放音也能收音
- **SPI 总线规划（说·亮点）**：「ESP32-S3 名义上四个 SPI，但两个被 Flash 和 PSRAM 占死了，真正能用的只有两个。屏要 40MHz 高速必须占一条硬件总线；SD 卡也要独立总线；触摸只要 2MHz 低速——所以我把触摸改成 GPIO 翻转模拟的软 SPI，把省下来的硬件总线让给 SD 卡。资源不够时按‘谁快谁占硬件’来取舍。」（为此自己写了软 SPI 到显示驱动接口的适配层）
- **存储（说）**：16MB Flash 我划了六个分区：参数区 20KB、引导数据区 8KB、两个 6.25MB 的应用区（分区类型就是 ota_0 和 ota_1）、3.375MB 的资源区、64KB 的崩溃转储区；8MB PSRAM 在堆配置里开了“纳入 malloc 统一管理”——小于 16KB 的分配优先内部 RAM、内部固定保留 32KB；图形库的全屏双缓冲两块各约 150KB 放在 PSRAM
- **软件分层（说）**：三层。驱动层一个外设一个组件，头文件只露初始化和操作接口；中间层是 LVGL 官方适配组件、网络组件和我集成的音频混音库；应用层是十个界面加四个常驻应用任务（串口助手的任务是进页面才创建的）。任务间用全局状态结构体加标志位通信，谁碰界面谁先拿图形库的互斥锁
- **双核（说）**：四个应用控制任务绑核 0、优先级 1（轮询标志位的慢循环）；音频混音和解码绑核 1、优先级最高（10 和 9）；LVGL 渲染优先级 6、不绑核，哪里空跑哪里

🔧【源码依据】lcd_set.c（SPI 40MHz/ST7789/LEDC 背光 5kHz-13bit）/ touch_set.c（softspi + 自写 panel_io）/ partitions.csv（nvs 20K / otadata 8K / app0+app1 各 6.25M / spiffs 3.375M / coredump 64K）/ sdkconfig（CPU 160MHz、PSRAM Octal @80MHz、SPIRAM_USE_MALLOC ≤16KB 走内部 RAM、内部保留 32KB）/ task_set.c（绑核 Core0 / prio1）

2.4 功能列表（已核对源码·照着说）

- **LVGL 图形界面（10 屏，纯触摸）**：主页时钟+日历+应用图标；设置里有 WiFi 开关、扫描出来的网络列表、软键盘输密码、亮度滑条（拖动实时调背光）；天气页有当前天气、未来三天滚动列表、明天详情和手动刷新；音乐页有歌曲列表、播放暂停上下曲、单曲循环/顺序/随机三种模式、音量滑条；还有四款游戏、语音对话页、串口助手（进页面创建任务、退出就删）、日历（对时后自动标今天）和关于页。界面骨架用 SquareLine 拖的，事件逻辑全是自己写的；切屏时把旧屏删掉再建新屏，内存不白占
- **WiFi 配网 + NTP + 天气（说）**：配网是屏上直接操作——打开开关设备自己扫描，列表里点一个，软键盘输密码连上（**不是**热点配网也**不是**一键配网）；密码 WiFi 驱动自动存进 NVS，重启后调一次连接就用存好的配置自动连上（运行中断线目前不会自动重连——诚实欠账，见专题 C）；联网后 SNTP 对时（阿里云服务器，东八区）；天气用心知天气接口，HTTPS 拉回来 cJSON 解析，当前天气加未来三天预报。时间每秒刷，判断对时成功的土办法：未同步时系统时间停在 1970，年份对了就是同步上了
- **音乐播放（说）**：SD 卡扫出 MP3 文件，libhelix 解码成 PCM，经环形缓冲进混音器，I2S DMA 打给功放；音量是输出前对采样点做软件增益（满音量时零开销）；不同歌采样率不一样就重配 I2S 时钟；文件系统开了中文长文件名，中文文件名按 UTF-8 读出来没问题
- **小游戏（4 个）**：2048、羊了个羊、消消乐、植物大战僵尸——开源社区游戏集合做的组件化适配；游戏逻辑跑在 LVGL 上下文里（组件内部用 LVGL 定时器驱动，跟渲染同一任务——纯 LVGL 组件也不可能自己开线程），天然不用加锁

- **AI 对话 (说)**：用乐鑫的小智组件接 xiaozhi.me 云端。链路：PDM 麦 16k 采集→设备端 OPUS 编码→云端语音识别→大模型 (Qwen/DeepSeek) →语音合成回 OPUS→本地解码 48k→重采样 44.1k→和音乐共用混音器从喇叭出去。第一次用要 6 位验证码到官网激活；支持连续对话，它说完自动重新听；还挂了 MCP 协议引擎，能扩展成控制设备

🔧【源码依据】ui_set/screens/*.c (十个屏) / wifi_set.c (阻塞扫描→roller→连接；凭据 WIFI_STORAGE_FLASH 自动进 NVS；重启 esp_wifi_connect() 无参用存配置；WIFI_EVENT_STA_DISCONNECTED 只置状态标志=无自动重连) / data_set.c (SNTP POLL 模式阿里云 ×3 + setenv("TZ","CST-8"); 心知天气 now.json+daily.json, HTTPS + esp_crt_bundle + cJSON; 年份 ≥2026 判同步) / music_set.c (opendir 扫 .mp3; 音量=int16 乘 music_volume/100 用 int32 中间量; disable→reconfig_clk→enable) / esp-audio-player (libhelix 解码 + 每流 esp_ringbuf) / sdkconfig 实测 (CONFIG_FATFS_LFN_STACK=y + CODEPAGE_936=y + UTF_8 = 中文长文件名——文件名读取没问题; **屏幕中文字形是否覆盖全未上机验证, 别说满话**; LV_COLOR_16_SWAP=y SPI 字节序; MONTERRAT_48 大号时钟字体) / esp_lvgl_games (百问网+B站@老虎爱3D 游戏集合组件化) / esp_xiaozhi + micro-opus + websocket/mqtt 组件

2.5 分工与设计 (已核对源码·照着说)

- **分工口径 (说)**：个人独立项目，从核对引脚接线、驱动移植、任务划分到全部应用逻辑一个人完成。(如果面试官问"团队合作/协作经历"：别在这个独立项目里硬编故事，转到实习联调——比如和写上位机/服务端的同事事先约定好数据帧格式再各写各的，联调对不上就一起抓日志定位是谁的问题。原句"协作能力用实习联调经历补"就是这个意思)
- **设计理念 (说)**：高内聚低耦合——一个外设一个组件、接口只露初始化和操作，应用层不碰寄存器；界面和业务分离，回调里只改标志位、刷控件，业务全在后台任务；全局一个状态结构体当数据中枢；不用的界面切走就删、不用的任务退出就销毁，内存按需用
- **任务间通信 (说·诚实口径)**：全局状态结构体+标志位轮询+LVGL 互斥锁，没用队列和事件组——为什么这么做、代价是什么、生产上怎么改，专题A-3 有完整话术
- **对外模块清单 (说)**：自研十四个组件——十个业务组件在用 (显示/触摸/LVGL适配/界面/任务编排/串口/WiFi/数据/音乐/小智)，四个教程基础组件 (按键/RGB/ADC/定时器) 挂在工程里但上电流程没调用它们的初始化，相当于备而不用；+ 集成的一个 C++ 音频播放库 + 十来个乐鑫托管组件

🔧【源码依据】main/CMakeLists.txt 的 REQUIRES 直连 13 个组件：rgb_set / key_set / timer_set / adc_set / uart_set / lcd_set / touch_set / lvgl_set / ui_set / task_set / xiaozhi_set / music_set / wifi_set; data_set 挂在 task_set 的依赖里 (task_set/CMakeLists.txt: REQUIRES uart_set lcd_set wifi_set data_set music_set xiaozhi_set esp_lvgl_games; data_task 由 task_init 创建 6KB) ; app_main (55 行逐行核实) 未调用 key/rgb/adc/timer 的初始化；托管组件 lvgl 8.3.11、esp_lvgl_adapter、esp_lcd_touch_xpt2046、cjson、esp-libhelix-mp3、esp_xiaozhi、micro-opus、esp_lvgl_games、esp_websocket_client、mqtt

2.6 技术专题深挖 (每专题=预判问题+第一人称回答话术; A~F 全部就位)

专题A：双核与任务划分 (已对照16章工程源码核实)

核心事实：4 个应用任务 wifi/data/music (6KB 栈)、xiaozhi (8KB 栈) 优先级全 1，绑 Core0；音频混音 prio10、解码/TTS 流 prio9 绑 Core1；LVGL 任务 prio6/8KB 栈/不绑核/心跳 1ms；串口任务 5KB 进页面建、退页面删；主任务 3584B。

A-1 (🔥 必问) "双核怎么分工的?"

「ESP32-S3 是双核的。我的分工：WiFi 协议栈是乐鑫默认绑在核 0 的，我不动；应用层四个控制任务——WiFi、时间天气、音乐控制、语音——我也绑到核 0，优先级只给 1，因为它们就是 200 毫秒到 1 秒轮询一次标志位的慢循环，很轻，被协议栈抢走 CPU 也无所谓；然后把最吃实时性的音频绑到核 1——混音任务优先级 10，解码和 TTS (Text To Speech, 文字转语音) 两个流任务 9——I2S 是等时性外设，供数一断就爆音，让它独占一核最稳；LVGL 渲染任务优先级 6，不绑核，两个核哪边空跑哪边。一句话：核 0 管系统加网络加轻控制，核 1 专职音频，图形渲染浮动。」

- **追问 L2：为什么这么分？**「WiFi 协议栈在核 0 是 IDF 的默认行为，主函数也在核 0，轻控制任务放它旁边顺水推舟；音频最怕打断，单独一核给最高优先级；LVGL 负载是弹性的——脏区大就重、小就轻，绑死一个核反而可能撞上音频，浮动最灵活。」
- **追问 L3：绑核怎么实现？绑错了会怎样？**「创建任务时多传一个核号参数就行，不绑就传'不限制'。绑错的典型现象：音频绑到核 0，网络一大流量 I2S 就欠载爆音；所有任务堆一核、另一个核闲死；同优先级绑同一核只能时间片轮转，分两核才真并行。」
- **犀利追问①"四个任务全绑核 0，WiFi 任务优先级 23（乐鑫默认就是这个数），不怕被饿着？"**「会周期性被打断，但这四个任务对时延完全不敏感，晚几毫秒没有影响；真正有实时要求的已经挪去核 1 了——这是拿'不敏感任务容忍抖动'换架构简单。」
- **犀利追问②"混音任务凭什么 10，比 LVGL 的 6 还高？"**「音频是流式等时负载，欠一帧立刻爆音可听见；图形晚一帧只是慢一点。等时性大于交互性，这是嵌入式多媒体系统的通用取舍。」

🔧【源码依据】task_set.c: task_create() 里 xTaskCreatePinnedToCore(函数, 名字, 栈, 参数, 优先级, 句柄, 核号), 不绑核传 tskNO_AFFINITY(-1); 音频库 mixer 配置 coreID=1/priority=10, stream priority=9/coreID=1; IDF 栈参数单位是**字节** (StackType_t=uint8_t, 原生 FreeRTOS 才按字)——教程写"字"是笔误, 面试指出是加分点

A-2 (◆) "任务列表？优先级怎么排？栈多大、怎么定的、溢出怎么发现？"

「全部报得出：LVGL 渲染 8KB、优先级 6、不绑核；音频混音优先级 10、解码和 TTS 流 9，都在核 1；应用四个任务——WiFi、时间天气、音乐各 6KB，语音 8KB，优先级全 1、绑核 0；串口助手任务 5KB，进串口页才创建、退出就删；主任务默认 3584 字节。栈怎么定：先按经验给初值，烧进去跑，用 FreeRTOS 的栈水位接口看每个任务历史最低剩余，再收放。语音任务给 8K 是因为任务栈上放了 2KB 的麦克风缓冲；天气任务 6K 里还塞了 2.5K 的请求和响应缓冲，贴着边——这是我知道要再留余量的点。溢出两层防护：IDF 默认带栈溢出检测——任务栈底的哨兵魔数被踩直接 panic 并打印任务名；运行期看栈水位；分区里还留了 64KB 崩溃转储区，真崩了能回读现场。」

🔧【源码依据】uxTaskGetStackHighWaterMark (栈水位)；栈溢出 canary 检测 (FreeRTOS 配置项, panic 打印任务名)；coredump 分区 0x10000；xiaozi_task 栈上 uint8_t pdm_buf[2048]；data_task 栈上 char url[512]+buf[2048]

A-3 (◆) "任务间通信用了什么？为什么不用队列？"

「三种：①一个全局应用状态结构体——分 WiFi、数据、音乐、语音四块——加布尔标志位，后台任务 200 毫秒到 1 秒轮询，开关型命令这样最直观；②LVGL 非线性安全，所以适配层提供了互斥锁，任何非图形任务碰控件前必须先拿；③音频库内部每个流一个环形缓冲，解码任务往里塞、混音任务往外取。没用队列和事件组，实话。代价我清楚：轮询有最高 200ms 延迟；标志位是普通布尔没加 volatile——但每个标志实际只有单一写者：按钮的写发生在 LVGL 任务的事件回调里，WiFi 状态的写是事件任务，单写多读的布尔读写是原子的，轮询循环每圈又重新加载，实际没出过问题；规范写法应该加 volatile 或者干脆改队列，这是代码规范上的欠账，我认。生产化改法：命令走队列——线程安全、没有轮询延迟；多条件同步用事件组一次等齐'WiFi+时间+天气'都就绪；任务唤醒用任务通知，延迟最低。」

- **追问"UI 回调跑在哪个上下文？"**「LVGL 任务的定时器上下文里。所以所有按钮写标志位其实是同一个任务在写——这也是'单写者'能成立的原因。」
- **追问"锁拿太久界面不就卡死？"**「我只在'改控件'的临界区拿锁，都是毫秒级；真正耗时的事——HTTP 请求、SD 卡扫描、音频解码——全在锁外做，锁里只做改文本、设选项这种快操作。」

🔗【源码依据】app_set_t (uart/setting/data/music/xiaozhi 五块, task_set.h) ;
esp_lv_adapter_lock/unlock; esp_ringbuf; 改进方案对应 xQueueSend / EventGroupWaitBits /
xTaskNotify (现在没用, 是"如果重做"的答案)

A-4 (○) "看门狗有没有用? 任务饿死怎么防?"

「IDF 默认开中断看门狗和任务看门狗, 任务看门狗默认订空闲任务。我的任务优先级虽低, 但每个循环都有延时让出 CPU, 空闲任务能跑, 狗就有人喂, 从没触发过; 真有任务死循环不让出, 会打印看门狗触发的日志加被怀疑的任务名。需要的话还有现成接口把关键任务显式订阅进看门狗、循环里喂狗。饿死预防靠三点: 优先级分层——音频 10、图形 6、控制 1; 音频和图形都是'干一下歇一下', 不长期占核; 同优先级 FreeRTOS 默认时间片轮转, 系统节拍我配的 1000 赫兹。」

🔗【源码依据】中断看门狗+任务看门狗 (IDF 默认开, 监控 IDLE0/1) ; esp_task_wdt_add /
esp_task_wdt_reset (显式订阅, 项目未用) ; CONFIG_FREERTOS_HZ=1000; 所有任务循环含
vTaskDelay

专题B: LVGL 移植全流程 (✅ 已写 · 全部对照源码核实)

核心事实: lvgl 8.3.11 (UI 骨架由 SquareLine 按 8.3.6 生成) ; LVGL 任务由适配层创建
(8KB/prio6/不绑核/1ms 心跳, 默认 33ms 刷新周期≈30帧上限) ; 显示=SPI+PSRAM 方案: 全
屏高双缓冲 2×150KB 放 PSRAM; 16 位色+字节序交换宏; 触摸=XPT2046 (软 SPI) 注册成输入
设备; 线程安全=适配层互斥锁; 性能角标未开 (要用再开) 。

B-0 (🔥 必问) "LVGL 是怎么移植到这个工程里的? 完整步骤讲一下"

「原理一句话: LVGL 是纯软件画图库, 不认识任何硬件——它只会喊'我要在某个坐标画一块颜色'。移植就是替它接好三根线: **给眼睛**——像素怎么送上屏; **给手**——手指按了哪里; **给心跳**——定时喂节拍、周期调刷新, 它才活起来。前两根线就是之前点亮的 LCD 驱动和触摸驱动; 乐鑫的适配组件相当于官方把三根线焊好的'转接头'。

实际动手五步: **第一步拉依赖**: 一条命令给工程加适配组件, 它连带把 LVGL 本体拉下来; 依赖清单文件是源头、要提交 git, 最终工程里我把 LVGL 锁在 8.3.11。 **第二步改三个构建文件**: 主组件用公共依赖 (主程序要直接调函数), LVGL 适配的代码用私有依赖 (只在内部用); 这里有个坑——依赖名必须写带命名空间的完整真名 (espressif_ 前缀), '写短名报'找不到文件', 报错完全看不出是名字写错。 **第三步改两个驱动组件存句柄**: LCD 组件把面板 IO 句柄也存进设备结构体、触摸组件把触摸句柄存进结构体——原来句柄是局部变量, 函数一返回就丢, 而 LVGL 注册时要用。 **第四步写 LVGL 适配代码**, 就四步: 初始化适配器、把 LCD 句柄按'SPI 加 PSRAM 全屏双缓冲'方案注册成显示设备、把触摸句柄注册成输入设备、启动适配器任务——之后心跳和刷新它全自动跑。 **第五步配置加主流程**: 工程默认配置开 16 位色 (不开的话默认 32 位色跟 RGB565 屏对不上, 颜色全错)、系统节拍 1000 赫兹、自定义分区表 (图形库加界面固件超了默认 1MB 应用区); 上电顺序就是依赖顺序——点亮屏、初始化触摸、启动 LVGL, 然后**拿着适配器的锁建界面**。」

- **追问"显示那根线深处是怎么工作的?"**「LVGL 每画完一块脏区就回调刷屏函数, 函数里调显示驱动的'画一块位图'接口把像素推给 SPI, 推完报告'这块画完了', 它才画下一块——双缓冲时上一块的传输和下一块的渲染是并行的。」
- **追问"为什么 ui_init 要包在适配器锁里?"**「界面初始化会创建控件、加载首屏, 全是 LVGL 操作; 而那时适配器的渲染任务已经跑起来了, 两边并发碰 LVGL 会出事——所以初始化也守同一把锁的规矩。」

四个必背坑 (被追问"你踩过什么坑"时甩出来, 证明真做过) :

1. 依赖名少了命名空间前缀 → 编译报找不到文件, 且报错看不出是名字问题;
2. 16 位色没开 → 默认 32 位色对不上 RGB565 屏, 颜色全错;
3. 面板 IO 句柄忘存进结构体 → 适配器永远等不到'这块画完'的通知, 屏亮了但永不刷新;

4. 碰 LVGL 不拿锁 → 多任务数据竞争，随机花屏甚至崩溃。

🔧【源码依据】main/idf_component.yml (lvgl/lvgl ^8.3.11、espressif/esp_lvgl_adapter、atanisof/esp_lcd_touch_xpt2046 ^1.0.6、espressif/cjson、espressif/esp_xiaozhi ^0.1.1、esphome/micro-opus ^0.4.1、lee-stone/esp_lvgl_games——组件管理器拉取)；lvgl_set/CMakeLists.txt (PRIV_REQUIRES espressif__esp_lvgl_adapter lcd_set touch_set——私有依赖+命名空间真名)；lcd_set.h (lcd_dev_t = panel + panel_io 两句柄)；touch_set (dev->handle)；lvgl_set.c 25 行四步；教学笔记第10章 sdkconfig.defaults (COLOR_DEPTH_16/PERF_MONITOR/FREERTOS_HZ=1000/自定义分区表，教学版锁 v8.4) vs 最终工程 manifest 锁 ^8.3.11——版本口径以最终工程为准；main app_main 顺序 (lcd→touch→lvgl→uart→wifi→lock+ui_init+unlock→music→xiaozhi→task_init)

B-1 (🔥 必问) "LVGL 移植你具体做了哪些事？"

「LVGL 本身是个纯 C 的图形库，它不知道硬件长什么样，移植就是给它接三样东西：显示、触摸、时间基准。显示：我先用 LCD 驱动把屏点亮，拿到屏的句柄；然后把句柄交给乐鑫官方的适配组件，它内部完成了渲染缓冲管理和刷屏对接——具体说就是注册一个刷屏回调，LVGL 每画完一块脏区就调它，回调里调显示驱动的画图接口把数据推给 SPI。触摸：XPT2046 驱动初始化完，把句柄也交给适配组件，注册成输入设备，LVGL 自己周期性读坐标、自己处理按下抬起。时间基准：适配组件内部用系统定时器给 LVGL 供毫秒心跳，动画才走得动。所以我自己动手的是：点亮屏、写触摸驱动、选缓冲方案、定渲染任务和锁；标准化的胶水适配组件包了。最后把 SquareLine 拖出来的十个界面骨架接上我手写的事件逻辑。」

- **追问 L2：显示缓冲怎么选的？**「三种方案：十分之一屏单缓冲最省内存但渲染和传输完全串行；全屏单缓冲居中；全屏双缓冲——CPU 渲染下一块时 DMA 还在推上一块，流水线并行最流畅。我内部 RAM 紧张但 PSRAM 有 8MB，所以选全屏双缓冲放 PSRAM，两块各 150KB。代价是 PSRAM 走八线 80MHz 的 SPI，带宽比内部 SRAM 低，但撑 30 帧绰绰有余。」
- **追问 L3：颜色为什么有字节序问题？**「RGB565 一个像素两个字节，LVGL 在内存里按小端序放，SPI 屏期望大端序收，高低字节要换。两种做法：发送前 CPU 逐像素换，费 CPU；或者配一个宏让 LVGL 渲染时就按换好的顺序放——我用后者，零额外开销。」
- **追问 L4：多任务下怎么保证线程安全？**「三条规则：①UI 操作尽量都发生在 LVGL 任务自己的上下文——按钮回调天然就在里面；②别的任务非要碰 UI，比如音乐任务要更新歌名，先拿适配层的互斥锁；③锁里只做改文本、设列表这种毫秒级快活，HTTP、SD 卡、解码这些重活全在锁外。另外切屏时我先删旧屏再建新屏，控内存峰值。」
- **追问 L5：卡顿怎么排查？**「先把性能角标打开，屏上直接显示 CPU 占用和帧率；再查四样：回调里有没有干重活（有就扔给后台任务）；锁有没有被长期占着；脏区是不是异常大（整屏频繁重画）；还有主频和缓冲配置。」
- **犀利追问“为什么不自己写 flush 回调，要用官方适配组件？”**「自己写完全可行——就是把脏区数据送到显示驱动画图接口，再加 tick 和缓冲管理。官方适配组件把这块标准胶水写好了，还顺带把 LVGL 任务、锁、触摸输入都管了，我用它省出时间写业务；但我清楚它里面每层在干什么，出问题我能剥开看。」

🔧【源码依据】lvgl_set.c 全文 25 行 (esp_lv_adapter_init → ESP_LV_ADAPTER_DISPLAY_SPI_WITH_PSRAM_DEFAULT_CONFIG 注册显示[宏展开=buffer_height 全屏高+双缓冲] → ESP_LV_ADAPTER_TOUCH_DEFAULT_CONFIG 注册触摸 → esp_lv_adapter_start 起任务)；适配层头文件 (任务 prio6/8KB/不绑核/1ms tick)；CONFIG_LV_COLOR_16_SWAP=y；LV_DEF_REFR_PERIOD 默认 33ms；esp_lv_adapter_lock/unlock(-1)=死等；esp_lv_adapter_get_fps；CONFIG_LV_USE_PERF_MONITOR (当前未开，menuconfig 可开)；ui.c (SquareLine 骨架+手写事件回调)

专题C: WiFi 配网 / NVS / NTP / 天气 (✅ 已写 · 全部对照源码核实)

核心事实: STA 模式屏上配网: 开关→阻塞扫描→roller 选 SSID→软键盘密码
→set_config+connect; 凭据 WIFI_STORAGE_FLASH 自动存 NVS; 重启无参 connect 用存配置;
事件回调 (STA_START/DISCONNECTED/GOT_IP) 只置 app 状态标志; **运行中断线无自动重连**
(欠账); SNTP=阿里云×3 POLL+东八区; 同步判据=年份≥2026; 天气=心知
now.json+daily.json (HTTPS+证书包+cJSON), 响应缓冲 2048 可能截断 (已知瑕疵)。

C-1 (💡 必问) "配网流程讲一下, 为什么不用 AP 配网/一键配网?"

「我这是带屏设备, 配网直接在屏上做, 体验最好: 设置里打开 WiFi 开关, 后台起一次扫描, 把扫到的热点列在下拉列表里, 选中、弹软键盘输密码、连接。密码交给 WiFi 驱动后它自动存进 NVS——Flash 上的键值存储——下次上电我只要调一下连接接口, 它自己拿存好的配置连。为什么不用 AP 配网? 那是给没屏的设备准备的: 手机先连设备的热点再把密码发给它, 我有屏没必要绕这一圈。一键配网靠抓空气里广播包的长度规律编码密码, 受干扰影响成功率, 更不该用。」

- **追问 L2: 扫描会不会卡界面?** 「扫描接口可以选阻塞模式, 我用的是阻塞, 但它跑在 WiFi 任务里、优先级 1, LVGL 在另一个上下文照常渲染——界面不卡, 只是几秒后结果才出来。」
- **追问 L3: 断网了设备会自己连回来吗? (诚实题)** 「分两种: **重启后会**——因为凭据在 NVS, 上电调连接接口就用存好的配置; 但**运行中断线目前不会自动重连**, 断线事件只是把状态标志置成断开。这是我承认的欠账, 标准做法是在断线事件里做延时自动重连加指数退避, 连上后再补一次对时和天气刷新。」
- **追问 L4: NTP 怎么用? 怎么知道时间同步上了?** 「起 SNTP 客户端, 轮询阿里云的三台 NTP 服务器拿 UTC 时间, 本地设了东八区, 显示自动加 8 小时。判断同步: 没同步时系统时间从 1970 起跳, 我判断年份大于等于 2026 就当同步好了——土但可靠, 因为现在真实时间必然超过 2026。」
- **追问 L5: 天气数据流? JSON 解析注意什么?** 「HTTPS GET 心知天气的现况接口和三天预报接口, 证书校验用 IDF 内置的证书包, 不用自己塞根证书; 回来 JSON 用 cJSON 解析进状态结构体再刷界面。两个注意点: cJSON 解析完必须释放节点, 否则泄漏; 我的 HTTP 响应缓冲是固定 2048 字节, 预报数据长了可能截断——已知瑕疵, 正确做法按 Content-Length 动态分配。」
- **追问 L6: NVS、SPIFFS、FATFS 三个文件系统为什么都用?** 「各干各的: NVS 是键值库, 带磨损均衡和掉电保护, 存配置和 WiFi 凭据这种小数据; SPIFFS 在 Flash 上放打包好的资源; FATFS 挂 SD 卡, 因为要放用户的 MP3、要和电脑互通。选型看三点: 数据多小、写入频不频繁、要不要跟 PC 交换。」

🔧 【源码依据】wifi_set.c (esp_wifi_scan_start 阻塞参; WIFI_STORAGE_FLASH 默认;
WIFI_EVENT_STA_START/DISCONNECTED + IP_EVENT_STA_GOT_IP 三个回调只写
app.setting.wifi_status; esp_wifi_connect 在 L114/L178, esp_wifi_disconnect 在 L129/L184/L190
——无重连逻辑) / data_set.c (esp_sntp 阿里云×3 POLL; setenv("TZ","CST-8"); 年份≥2026 判据;
心知 now/daily 接口+esp_crt_bundle+cJSON; buf[2048] 固定)

专题D: I2S 音频链路 (✅ 已写 · 全部对照源码核实)

核心事实: TX=I2S0 标准 Philips 44100/16bit/双声道 (BCLK41/WS42/DOUT40, 无 MCLK);
混音任务 prio10/核1, 输出回调里软件音量; 解码流、TTS 流 prio9/核1, 各带环形缓冲; PDM
麦=I2S0 RX 16k/16bit/单声道 (CLK38/DATA39), 任务栈上 2048B 缓冲、60ms 一帧; 随机模式有真 bug (见追问⑦)。

D-1 (💡 必问) "整条音频链路讲一遍"

「放音方向：SD 卡里的 MP3 文件，解码流任务用 libhelix 解成 PCM，写进这条流自己的环形缓冲；混音任务把所有活跃的流相加，通过输出回调推给 I2S 的 DMA，DMA 按位时钟节拍自动打给数字功放，喇叭就响了。录音方向：PDM 麦克风接在 I2S 控制器接收端，16k 采样 16 位单声道，任务每 60 毫秒取一次数据交给语音组件上传。关键设计是混音器：I2S 外设和喇叭只有一套，但音乐和 AI 语音两路都要出——混音任务把两路相加成一路，所以放着歌也能随时插话对话。」

- **追问 L2：I2S 三根线各是什么？为什么不用 MCLK？**「BCLK 位时钟，每一位数据一个脉冲；WS 字选，翻转一次切左右声道；DOUT 串行数据。MCLK 是给需要主时钟做内部滤波的 DAC 用的；我们这块数字功放不需要，三线就够，省一个脚。」
- **追问 L3：为什么必须 DMA？**「I2S 是等时性外设——44100 赫兹就是每秒 44100 组左右声道样本，早一分晚一分都不行。CPU 逐样本写必死：高优先级任务一跑就断流。DMA 把内存缓冲自动搬进 I2S，CPU 只管及时补缓冲。」
- **追问 L4：欠载什么现象？怎么防？**「DMA 缓冲空了，功放拿到的是旧数据或废数据，人耳听就是爆音咔哒声。防法三层：缓冲给足当蓄水池；混音和解码放最高优先级、独占核 1，不被网络图形抢；解码往环形缓冲里塞、混音按节奏取，两边解耦。」
- **追问 L5：音量怎么做？**「功放没接模拟音量控制，软件做：I2S 输出回调里对每个 16 位样本乘音量百分比。两个细节：用 32 位中间量防溢出；音量 100% 时直接跳过循环，零开销。」
- **追问 L6：MP3 为什么选 libhelix？换歌采样率不同怎么办？**「libhelix 是定点整数解码库，内存占用小、不依赖浮点，专为 MCU 设计，对比 libmad 那种浮点大内存方案更适合嵌入式。采样率：每首歌可能不同，混音器的时钟回调先把 I2S 通道停掉、按新采样率重配时钟、再启动，保证主时钟跟得上文件。」
- **犀利追问⑦"播放模式都实现了？有没有 bug？"（主动坦白是加分项）**「三种都实现了，但我审自己代码时发现随机模式有个真 bug：随机下标是对'歌曲数减一'取模——最后一首永远选不到，极端情况歌单只有一首时会变成对零取模直接崩。正确写法是对歌曲数取模。这是我知道、能讲清、知道怎么修的 bug。」

🔧【源码依据】music_set.c L113-148 (I2S std Philips 44100/16/2、mclk=UNUSED；audio_mixer_init priority=10/coreID=1/write_fn=audio_i2s_write/clock_set_fn；audio_stream_new "music" DECODER prio9/core1)；xiaozi_set.c L124-163 (PDM RX 16000/16BIT/MONO；tts RAW prio9/core1)；音量回调 (int32 中间量、vol==100 跳过)；随机 bug: esp_random()%(cnt-1) (L204)；pdm_buf[2048] 在 xiaozi_task 栈、60ms 帧读

专题E：Flash/PSRAM 内存分配 (✅ 已写 · 全部对照源码核实)

核心事实：16MB Flash QIO：nvs 20K / otadata 8K / app0+app1 各 6.25M (OTA 预留) / spiffs 3.375M / coredump 64K；PSRAM 8MB Octal @80MHz：LVGL 双缓冲 2×150KB、OPUS 解码两块缓冲约 34.5KB 显式放 PSRAM；堆策略=大于 16KB 自动落 PSRAM、内部保留 32KB 给关键路径；内部 SRAM 512KB 优先任务栈/DMA/协议栈。

E-1 (🔥 必问) "内存怎么规划的？"

「这颗芯片三层存储：片内 SRAM 512KB 最快；Flash 16MB 放代码和只读数据；外挂 PSRAM 8MB 当大内存扩展。我的原则是'快的给关键路径，大的给大缓冲'：任务栈、DMA 缓冲、协议栈这些必须内部 SRAM——堆我配成'小于 16KB 的分配优先内部 RAM，内部固定保留 32KB 不让大块侵占'；LVGL 的双缓冲约 300KB、语音解码的缓冲，这种上百 KB 的大块显式或自动落 PSRAM。Flash 侧：参数区、两个等大应用区 (OTA 预留)、资源区放字体图片、崩溃转储区。」

- **追问 L2：PSRAM 和内部 SRAM 差在哪？什么能放什么不能放？**「内部 SRAM 零等待直接访问；PSRAM 走八线 SPI 80MHz，带宽低一个量级，DMA 访问还有限制。所以实时关键的小缓冲绝不放 PSRAM；大块、访问频率相对低的才放——图形缓冲就是典型：一次推一大块，不敏感单次延迟。」

- **追问 L3：为什么 LVGL 缓冲必须放 PSRAM？**「两块全屏 150KB 共 300KB，内部 SRAM 总共才 512KB，还要跑协议栈、任务栈、各种驱动——塞不下，塞下了系统也残废。PSRAM 8MB 富余，代价只是带宽，八线 80MHz 撑 30 帧够用。」
- **追问 L4：内存泄漏和碎片怎么查、怎么防？**「泄漏：运行期定期采样剩余堆和历史最低水位，看趋势往下走就是漏；IDF 还有堆任务跟踪，能定位到是谁分配没放。碎片：表现为'总剩余够但大块分配失败'；防法是大块缓冲启动时一次性分配好，运行期不反复 malloc/free——我的图形缓冲、音频缓冲、解码缓冲全是初始化时一次分配，常年复用。」
- **追问 L5：OTA 那两个分区讲讲？**「两个 6.25MB 等大应用区加一个引导数据区，这是 OTA 的标准布局：新固件写另一个区，引导数据区决定下次启动哪个，启动成功才更新指向——失败可以回滚。但我要诚实：目前只把分区划好了，OTA 升级流程本身还没写，这是我计划补的功能。」

🔧【源码依据】partitions.csv 全表；sdkconfig (CONFIG_SPIRAM Octal 80MHz；SPIRAM_USE_MALLOC=y；MALLOCC_ALWAYSINTERNAL=16384；MALLOCC_RESERVE_INTERNAL=32768)；LVGL 双缓冲=适配层宏 (buffer_height=240、require_double_buffer=true)；xiaozi_set.c L89-90：opus_buf=heap_caps_malloc(57602, MALLOCC_CAP_SPIRAM)、out_buf=heap_caps_malloc(57604, MALLOCC_CAP_SPIRAM)

专题F：AI 对话链路 (✅ 已写 · 全部对照源码核实)

核心事实：乐鑫 esp_xiaozi 组件 + xiaozi.me 云 (WebSocket 承载音频/对话，MCP 引擎挂设备控制)；上行=PDM 16k 单声道→组件端 OPUS 编码上传 (对话状态机"对话中"时每 60ms 读麦)；云端 ASR→LLM (Qwen/DeepSeek)→TTS；下行=OPUS 48k 单声道 (约 3~5KB/s)→本地 opus_decode (单帧上限 5760 样本)→按下标映射重采样 44.1k→复制双声道→TTS 流 prio9/核1→混音器共喇叭；激活=屏显 6 位码→官网绑定 (任务每秒轮询激活状态，未激活时禁用对话开关)；启动/停止=开关置状态位、任务轮询调组件的 start/stop；会话内组件自动管理"聆听-思考-说话"切换。

F-1 (🔥 必问) "语音对话整条链路讲一遍"

「用乐鑫的小智组件接小智云服务。上行：PDM 麦克风 16k 单声道采集——原始 PCM 是每秒 32KB——组件在设备端用 OPUS 压到每秒几 KB，通过 WebSocket 长连接推给云端；云端先语音识别转文字，再给大语言模型生成回答，再语音合成回 OPUS 音频。下行：我本地把 OPUS 解成 48k 的 PCM，重采样到 44.1k、复制成双声道，写进一条 TTS 音频流——它和音乐流在混音器里汇合共用喇叭。对话文本通过事件回调拿互斥锁刷到屏幕，问题、回答各一个文本框。开关状态机：用户打开对话开关，任务轮询到状态变化就调组件的启动接口，通道建立后组件自动发唤醒词'你好小智'开始会话。」

- **追问 L2：为什么 OPUS？为什么 WebSocket 不用 HTTP？**「OPUS 是语音场景里码率和延迟最平衡的编码，16k 采样从每秒 32KB 压到几 KB，设备端软解算力也扛得住。HTTP 是请求-响应模型，语音对话要持续双向推流，WebSocket 长连接全双工天然合适。」
- **追问 L3：本地离线唤醒吗？(诚实题)**「不是。本地离线唤醒要额外的关键词检测模型和算力，我的做法是：用户打开对话开关，通道建立时由组件发唤醒词文本'你好小智'开始会话，TTS 播完自动回到聆听状态形成连续对话，用户关开关就停。产品化才需要上本地 KWS。」
- **追问 L4：激活绑定怎么做的？**「首次连上服务器会下发 6 位验证码，我把码显示到屏幕文本框、把对话开关禁用置灰；用户到官网注册后输入这六个数字绑定设备；我的任务每秒查一次激活状态，查到激活才放开开关，进入正常对话。」
- **追问 L5：重采样怎么做的？会不会有音质损失？**「48k 转 44.1k，我用的是最简单的按比例丢点映射：输出第 j 个样本取输入第 j 乘 44100 除 48000 个样本，再复制成双声道。优点是零乘加开销；缺点是没做插值，高频上有一点量化损失——语音场景听不出来。要更好可以上线性插值，代价是每样本两次乘加。」

- **犀利追问"这方案的弱点?"**「四个：强依赖网络，断网对话直接不可用；延迟在云端往返，本地压不了太多；语音全上云，产品要处理隐私合规；硬件上没有回声消除——喇叭放音时麦克风会拾到自己的声音，真全双工要加 AEC。这些都讲得出，说明我知道这方案边界在哪。」

🔗【源码依据】xiaozi_set.c 全文 (L89-90 opus 缓冲 PSRAM; L101 opus_decode 上限 5760 样本; L108-119 重采样 $j=i*44100/48000$ +双声道复制+audio_stream_write_pcm(tts_stream,...,100ms); L124-137 PDM RX 16000/16BIT/MONO; L140-150 esp_mcp_create+chat_cfg(mcp_engine, owns_mcp_engine=true)+esp_xiaozi_chat_init; L165-223 任务：激活轮询 1s/状态 1=chat_start 2=chat_stop 3=对话中 60ms 读麦 esp_xiaozi_chat_send_audio_data) ; esp_xiaozi 组件 (WS+mqtt+唤醒词文本+会话状态机) ; micro-opus; 教程 15 章 (云端 LLM=Qwen/DeepSeek, TTS 约 3~5KB/s)

2.7 项目成果（已核实·照着说）

- LVGL 8.3 集成完成（界面骨架由 SquareLine 生成、工程锁定 lvgl 8.3.11），十个界面流畅运行；帧率实测手段：在配置里打开 LVGL 自带性能角标（perf monitor），屏上直接显示 CPU+FPS，或调适配层取帧率接口【需上机实测一个真数】
- WiFi 屏上配网 → SNTP 对时 → 心知天气当前+三天预报，全链路跑通
- MP3 三种播放模式（中文歌名正常显示）+ 四款 LVGL 游戏 + 语音 AI 对话（可与音乐共用喇叭混音输出）全部可用
- 规模：自研 14 个组件（10 个业务在用 + 4 个教程基础组件挂在工程、未调用初始化）+ 集成 1 个 C++ 音频库 + 约 10 个托管组件；常驻任务 9 个；分区表六个分区（含两个 OTA 类型应用区，升级流程未实现）+ 崩溃转储区
- 简历口径就按本节和本节开头那段"简历项目描述改稿"写，改简历直接抄
- 【待补充：实测帧率/内存水位/稳定性时长，上机一次测掉】

2.8 模拟面试考核记录（2026-09-04 改版：不再写答题卷）

流程：你通读 2.1~2.6 → 逐节脱稿复述（建议手机录音，卡壳处回炉）→ 自觉差不多时说「开始考」→ AI 口头抽题+追问链刨根问底 → 每次考核往下面表里追加一行。

日期	抽的题	结果	卡壳点 / 回炉情况
(未开始)			

复述草稿存档（你 9-4 亲笔写的两段，留作对照进步）：

- 30秒版复述：「这个项目是我独立实现的智能终端设备，基于ESP32-S3的双核芯片，外挂16MB的闪存和8MB的内存，实现了一个完整的嵌入式产品形态，LVGL图形界面可以显示天气时间，WIFI联网之后会自动同步NTP时间，获取天气的API并且解析JSON渲染到屏幕上面去，在本地采用SPI去读取SD卡里面的MP3，再采用I2S去输出音频播放，还集成了ai语言交互，该项目采用的是FreeRTOS，双核分工，LVGL完整移植到SPI屏幕，显示缓存，触摸对接，多任务线程安全管理，是我将显示驱动，网络，音频，多任务调度技能合在一块的项目」
- 分工复述：「此项目是我独立完成的，核对引脚，LVGL的移植，任务的调度都由我一人完成」

三、项目2：ROS2 导航小车（排队中，ESP32 过关后做）

简历：树莓派4B + ROS2 Jazzy + STM32F103；STM32读MPU6050/编码器测速/PID闭环，自定义串口协议通信；树莓派里程计推算+话题+TF；Cartographer 2D SLAM；Navigation2 自主导航避障。

模板同项目1（2.1~2.8 结构）。专题预划：A上下位机分工与串口协议 / B编码器+MPU6050+PID / C里程计推算与TF / D Cartographer选型与建图 / E Navigation2导航与避障。

四、实习：串口转 WiFi 模块（排队中）

简历：STM32F407+ESP8266+FreeRTOS；AT命令联网/TCP；AT封装标准Socket接口；DMA+IDLE解决高波特率丢包；四层架构+队列信号量；多客户端并发验证；串口锁冲突/粘包；Modbus RTU/libmodbus；V853 V4L2。

专题预划：A四层架构 / B AT命令与Socket封装 / C DMA+IDLE丢包（必问） / D 粘包与串口锁 / E 多客户端并发 / F Modbus与libmodbus / G V4L2。

五、项目3：工业互联设备管理系统（排队中）

简历：STM32H563RI；libmodbus读写传感器+主控升级传感器固件；改写libmodbus底层+写文件块；点表机制；BootLoader IAP；USBX虚拟串口。

专题预划：A系统架构与Modbus选型 / B libmodbus改写 / C 点表机制（亮点） / D BootLoader/IAP六连问 / E USBX。

六、统一问题（非技术12问+反问，排队中）

优缺点 / 成就与挫折 / 压力最大 / 加班观 / 提前实习 / 团队冲突 / 学习方法 / 职业规划 / 企业项目vs自己项目 / 怎么看公司发展 / 反问环节。PDF 5.2.5 原题+老师模板口径。